CONCURRENT PROGRAMMING

CT074-3-2 CCP

INDIVIDUAL ASSIGNMENT

Name	**Low Yi Bo**
TP Number	**TP076096**
Lecturer	**TS. DR. MAYTHEM KAMAL ABBAS AL-ADILEE**
Intake	**APD2F2506CS(DA)**
Date Assigned	**11th December 2025**
Submission Date	**8th March 2026**

Table of Contents

1.0 Introduction and Background 3

1.1 System Overview 3

1.2 System Assumptions 3

2.0 Basic Requirements Met 4

2.1 Runway Mutual Exclusion Implementation 4

2.2 Ground Capacity Limitation Strategy 5

2.3 Aircraft Lifecycle & Passenger Concurrency 6

2.4 Simulated Operational Delays 9

2.5 ATC Ground Deadlock Prevention 10

2.6 Thread-Safe Statistical Tracking 11

3.0 Additional Requirements Met 12

3.1 Shared Refuelling Truck Resource 12

3.2 Congestion & Emergency Prioritization 13

4.0 Conclusion 15

5.0 Appendix 16

1.0 Introduction and Background

1.1 System Overview

The Asia Pacific Airport simulation is a model of the simultaneous operations of a restricted capacity airport. The facility has one common runway, three docking gates, and one refuelling truck. The system safely coordinates the landings, coasting, docking, disembarking, cleaning, refueling, boarding, and take-off of six airplanes with up to 50 passengers each as each makes requests.

A very important safety limit limits capacity in the ground to only 3 airplanes at any given time in order to prevent physical collisions. Furthermore, the system must handle complex, congested situations like an emergency landing without fuel, where the plane must be given priority over waiting planes. This simulation has a good case of thread safety, preventing deadlocks, and avoiding thread starvation with the help of concurrent manual java synchronization constructs.

1.2 System Assumptions

To simulate the airport environment effectively, the following operational and technical assumptions were applied during development:

Assumption	Description
Time simulation scaling	Real-world action delays are scaled down by using `Thread.sleep()` to ensure the entire simulation completes within the 60 second limit.
Passenger readiness	Passengers are always ready at the terminal, meaning there is no secondary delays during boarding or disembarking.
Emergency trigger	Planes do not dynamically consume fuel in the air. The emergency plane is explicitly flagged `isEmergency = true` upon creation to trigger priority handling.
ATC queueing fairness	ATC enforces strict FIFO queuing to prevent thread starvation. The emergency plane bypasses this rule to jump to the front of the queue.

Sequential refuelling	The single refuelling truck services all the plane sequentially. Concurrent requests place waiting planes into a blocked state until the truck is free.

2.0 Basic Requirements Met

2.1 Runway Mutual Exclusion Implementation

To achieve Basic Requirement 1, a single instance of Runway is shared by all of the threads of airplanes. Because the runway is a shared resource, it is a critical section where more than one plane trying to use it at the same time could cause errors or crash, also known as race conditions.

```java
public static void main(String[] args) throws InterruptedException {

    // initialize shared resources
    runway = new Runway();
    airport = new Airport();
    gates = new Gate[gatesCount];
    cleaners = new Cleaning[gatesCount];
    cleanerThreads = new Thread[gatesCount];
```

Figure 1 Initialization of the Runway Object

In order to perform the strict mutual exclusion, ReentrantLock is being used. The occupy() method will be used to lock the runway and the release() method will be used to unlock the runway. The lock is initialized with a fairness policy new ReentrantLock(true) which implies the implementation of a strict First-In-First-Out (FIFO) queue. This fairness helps make sure that every plane gets their turn eventually, so that no single plane is forced to wait forever, in other words, it can be said that this brings about an interesting property called thread starvation. Safely, lock.unlock() is put into a finally block to ensure that the lock will always be unlocked even when an error occurs. This is basic rule in order to prevent the whole system to freeze up also known as deadlock.

```java
public class Runway {

    //lock that ensure plane user the runway in order
    private final ReentrantLock lock = new ReentrantLock(true);
    private Plane currentPlane;

    //secure the runway for a specific plane
    public void occupy(Plane plane){
        lock.lock();
        try {
            currentPlane = plane;
        } finally {}
    }
    //free the runway for the next plane in queue
    public void release(){
        try {
            currentPlane = null;
        } finally {
            lock.unlock(); //unlocked when the plane safely leaves
        }
    }
```

Figure 2 ReentrantLock Implementation in Runway Class

2.2 Ground Capacity Limitation Strategy

To prevent collisions, an Airport class puts a limit of capacity using a Semaphore(3, true). The acquire() method is used to block any incoming planes when it is full and release() frees space when the plane takes off. The fairness Flag works to form a FIFO queue, so that starvation is avoided. An AtomicInteger is used to keep track of currentOccupancy with the benefaction of an inherent thread safety compared to an int and therefore eliminates with an approach to race conditions during updating of the capacity, without the use of synchronized blocks.

```java
public class Airport {

    //maximum number of planes allowed in airport ground
    private static final int maxCapacity = 3;

    //semaphore controls real capacity
    private final Semaphore capacity = new Semaphore(maxCapacity, true);

    //thread safe counter to track the current plane count in airport
    private final AtomicInteger currentOccupancy = new AtomicInteger(0);

    // ATC calls this when granting landing clearance
    public void planeEntered() throws InterruptedException {
        capacity.acquire();              // block if full
        currentOccupancy.incrementAndGet();
    }
    // Plane calls this when fully leaving airport
    public void planeLeft() {
        currentOccupancy.decrementAndGet(); //release the permit to wake up waiting threads
        capacity.release();
    }

    //getter
    public int getCurrentOccupancy() {
        return currentOccupancy.get();
    }
    public int getMaxCapacity() {
        return maxCapacity;
    }

    //condition checker
    public boolean hasSpace() {
        return capacity.availablePermits() > 0;
    }
    public boolean isFull() {
        return capacity.availablePermits() == 0;
    }
```

Figure 3 Airport Capacity Control Using Semaphore

2.3 Aircraft Lifecycle & Passenger Concurrency

To follow the Basic Requirement 3, the lifecycle of every airplane is executed sequentially in the run() method.

For the gate operations the system is optimized by using 1 thread to represent the whole group of passengers instead of generating a thread for each one of the passengers.

As an important mechanism for ensuring safety, Thread.join() method is used, to make sure these passenger threads are under control. join() is used to block the main plane thread safely, until the passenger group thread completes the task. This strictly controlling of the thread ensures safe operation from race conditions ensuring the plane will never undock while passengers are still boarding / disembarking without involving any complex manual wait/notify locks.

```java
public void run(){
    try{
        arrivalTime = System.currentTimeMillis();
        requestLanding();
        land();
        coastToGate();
        dock();
        performGateOperation();
        undock();
        coastToRunway();
        takeOff();

        Main.statistics.recordPlane(this);

    } catch(InterruptedException e){
        System.out.println(planeName + " was interrupted.");
        Thread.currentThread().interrupt();
    }
}
```

Figure 4 Plane run() method

```java
private void requestLanding() throws InterruptedException{
    //print the emergency tag
    if (fuelShortage) {
        System.out.println(planeName + ": Requesting Landing. [EMERGENCY]");
    } else {
        System.out.println(planeName + ": Requesting Landing.");
    }

    assignedGate = Main.atc.requestLandingPermission(this);
    landingTime = System.currentTimeMillis();
}
```

Figure 5 requestLanding()

```java
private void land() throws InterruptedException{
    System.out.println(planeName + ": Landing.");
    Main.runway.occupy(this);
    Thread.sleep(1000);
    System.out.println(planeName + ": Landed.");
}
```

Figure 6 land()

```java
private void coastToGate() throws InterruptedException{
    System.out.println(planeName + ": Coasting to Gate-" + assignedGate.getGateId());
    Main.runway.release();
    Thread.sleep(500);
}
```

Figure 7 coastToGate()

```java
private void dock() throws InterruptedException{
    assignedGate.dock(this);
    System.out.println(planeName + ": Docked at Gate-" + assignedGate.getGateId());
}
```

Figure 8 dock()

```java
private void performGateOperation()throws InterruptedException{
    // 1. unload arrival passengers
    disembark();

    // 2. concurrent services
    System.out.println(planeName + ": Requesting Cleaning at Gate-" + assignedGate.getGateId());
    assignedGate.requestCleaning(this); // Tell cleaners to start

    refuelPlane(); // Go get fuel at the same time

    assignedGate.waitForCleaning(); // Make sure cleaners are done before boarding

    // 3. load new departing passengers
    boardPassengers();
}
```

Figure 9 performGateOperation()

```java
private void undock() throws InterruptedException{
    System.out.println(planeName + ": Undock from Gate-" + assignedGate.getGateId());
    assignedGate.undock();
}
```

Figure 10 undock()

```java
private void coastToRunway() throws InterruptedException{
    System.out.println(planeName + ": Coasting to Runway.");
    Thread.sleep(500);
}
```

Figure 11 coastToRunway()

```java
private void takeOff() throws InterruptedException{
    System.out.println(planeName + ": Requesting Taking off.");
    Main.atc.requestTakeOffPermission(this);
    Main.runway.occupy(this);
    System.out.println(planeName + ": Taking-off.");
    Thread.sleep(1000);
    Main.runway.release();
    Main.airport.planeLeft();
}
```

Figure 12 takeOff()

```java
private void disembark() throws InterruptedException {
    Passenger disembarkingGroup = new Passenger(passengerCount, "Disembarking");

    Thread passengerThread = new Thread(disembarkingGroup, planeName);
    passengerThread.start();

    passengerThread.join();
}
```

Figure 13 disembark()

```java
private void refuelPlane() throws InterruptedException{
    System.out.println(planeName + ": Request refueling.");
    Main.refuelingTruck.refuel(this);
    System.out.println(planeName + ": Refueling completed.");
}
```

Figure 14 refuelPlane()

```java
private void boardPassengers() throws InterruptedException {
    this.passengerCount = random.nextInt(50) + 1;

    Passenger boardingGroup = new Passenger(passengerCount, "Boarding");

    Thread passengerThread = new Thread(boardingGroup, planeName);
    passengerThread.start();

    passengerThread.join();
}
```

Figure 15 boardPassengers()

2.4 Simulated Operational Delays

In order to fulfill the Basics of Requirement 4, some artificial time delays are enforced at each operating stage using Thread.sleep(). This is to ensure that actions like landing and refuelling don't happen instantaneously.

Implementing this method is justified over a busy-waiting loop, where the program continuously tests for updates inside a looping process, as it helps save the processing power of the computer. When sleep() method is called, it puts the current thread to a waiting state for some duration of time. This is a critical aspect from performance point of view because it allows the paused thread to make way for the computer to execute other active threads. Consequently, this system does not place a drain on processing power and it accurately emulates the real-world operational delays with no slowdown of the rest of the simulation.

```java
private void land() throws InterruptedException{
    System.out.println(Thread.currentThread().getName() + ":Landing.");
    Main.runway.occupy(this);
    Thread.sleep(1000);
    System.out.println(Thread.currentThread().getName() + ":Landed.");
}
```

Figure 16 Thread.sleep()

2.5 ATC Ground Deadlock Prevention

In order to meet the basic requirement number 5, the system employs a centralised controller for preventing ground waiting in the designated gate aircraft system. This controller is the central manager to make the threads safely coordinated. When the airplane requires the landing clearance, it invocations the requestLandingPermission method which puts the current airplane in a landing queue and safely waits for the thread using the wait method.

This implementation is justified as being an effective deadlock prevention strategy. Instead of allowing planes to land and block the airport grounds for days, hours, or days on end, the controller continually monitors all shared resources. It only provides clearance and wakes up a paused plane thread by the notify method both if physical runway and a gate slot are explicitly available. By making this a hard air holding pattern, the system ensures that there is safe allocation of resources and that planes will not get stuck waiting for each other in an endless loop, which completely avoids deadlocks in the system.

```
public class ATC implements Runnable {

    //queueing list for landing and takeoff sequence
    private final LinkedList<Plane> landingQueue;
    private final LinkedList<Plane> takeoffQueue;
```

Figure 17 ATC Landing and Takeoff Queue

```
public void run(){
    try{
        arrivalTime = System.currentTimeMillis();
        requestLanding();
        land();
        coastToGate();
        dock();
        performGateOperation();
        undock();
        coastToRunway();
        takeOff();

        Main.statistics.recordPlane(this);
```

Figure 18 Plane Operational Sequence

```java
    public Gate requestLandingPermission(Plane plane) throws InterruptedException {
        synchronized (atcLock) {
            landingQueue.add(plane);
            atcLock.notifyAll();
        }
        synchronized (plane) {
            while (plane.getAssignedGate() == null) {
                plane.wait();
            }
            return plane.getAssignedGate();
        }
    }
```

Figure 19 Landing Request and Waiting Mechanism

```java
        if (!Main.airport.hasSpace() || gateSlots.availablePermits() == 0) {
            if (landingPlane.getPlaneId() != lastDeniedId) {
                System.out.println(Thread.currentThread().getName() + ": Landing Permission Denied for Plane=" + landingPlane.getPlaneId() + ". Airport/Gates Full.");
                lastDeniedId = landingPlane.getPlaneId();
            }
        }
        // runway status
        else if (!Main.runway.isAvailable()) {
            if (landingPlane.getPlaneId() != lastDeniedId) {
                System.out.println(Thread.currentThread().getName() + ": Landing Permission Denied for Plane=" + landingPlane.getPlaneId() + ". Runway Occupied.");
                lastDeniedId = landingPlane.getPlaneId();
            }
```

Figure 20 Landing Permission Control Logic

2.6 Thread-Safe Statistical Tracking

To meet basic requirement 6, the system keeps track of waiting times, cumulative passengers, and planes served. The printStatistic method is a thread safe approach to reporting these results. Since more than one thread is used for updating data, the system has a synchronized block with a statsLock. This is a very important aspect of safety that can only have 1 thread modifying the shared data at a time. This helps in avoiding race conditions and helps make sure that the final output is accurate and free from data corruption.

```java
    public void printStatistic(){
        synchronized(statsLock){
            System.out.println("SIMULATION STATISTICS");

            if(waitingTimes.isEmpty()){
                System.out.println("No data collected");
                return;
            }

            long maxWaitTime = Collections.max(waitingTimes);
            long minWaitTime = Collections.min(waitingTimes);
            double avgWaitTime = waitingTimes.stream().mapToLong(Long::longValue).average().orElse(0);

            System.out.println("Waiting Time Statistic:");
            System.out.println("Max:" + maxWaitTime);
            System.out.println("Min:" + minWaitTime);
            System.out.println("Avg:" + String.format("%.1f", avgWaitTime));

            System.out.println("Service Statistics:");
            System.out.println("Total planes served: " + totalPlanesServed);
            System.out.println("Total passengers boarded: " + totalPassengersBoarded);

            if (totalPlanesServed > 0) {
                double avgPassengers = (double) totalPassengersBoarded / totalPlanesServed;
                System.out.println("Average passengers per plane: " + String.format("%.1f", avgPassengers));
            }
        }
    }
```

Figure 21 Simulation Statistics Calculation

3.0 Additional Requirements Met

3.1 Shared Refuelling Truck Resource

In order to meet the additional requirement 1, an independent refuelling truck is used as a separate thread in the system to simulate real world operations. The main method creates 1 instance of the RefuelingTruck class and has it running as a thread. When an airplane thread needs service, it calls the refuel method in which access is taken with the help of a ReentrantLock. The reason for using the lockInterruptibly method is that it is more flexible than the synchronized block because in this way the threads can get interrupted while waiting for the resource. This implementation is a critical part of the safety in that only 1 plane can have the truck as its currentPlane at any given time. By enforcing this strict mutual exclusion, the system avoids the possibility that more than one aircraft tries to use the single refuelling resource at the same time, which would avoid race conditions and ensures thread safe sequential servicing.

```java
    public static void main(String[] args) throws InterruptedException {

        // initialize shared resources
        runway = new Runway();
        airport = new Airport();
        gates = new Gate[gatesCount];
        cleaners = new Cleaning[gatesCount];
        cleanerThreads = new Thread[gatesCount];

        for(int i = 0; i < gatesCount; i++){
            gates[i] = new Gate();
            // Initialize a cleaning team for each gate
            cleaners[i] = new Cleaning(gates[i]);
            cleanerThreads[i] = new Thread(cleaners[i], "Cleaner-Gate" + (i+1));
        }

        statistics = new Statistic();
        refuelingTruck = new RefuelingTruck();
        atc = new ATC();

        // start threads
        Thread atcThread = new Thread(atc, "ATC");
        atcThread.start();

        Thread truckThread = new Thread(refuelingTruck, "RefuelingTruck");
        truckThread.start();
```

Figure 22 Refuelling Truck Initialization

```java
public void refuel(Plane plane) throws InterruptedException {
    truckLock.lockInterruptibly();
    try {
        // 1. Hand off the task
        this.currentPlane = plane;
        this.isRefueling = true;

        // 2. Wait for the Truck Thread to finish the work
        while (this.isRefueling) {
            Thread.sleep(50);
        }
    } finally {
        truckLock.unlock();
    }
}
```

Figure 23 Refuelling Control Mechanism

3.2 Congestion & Emergency Prioritization

To satisfy additional requirement 2, the system simulates a busy scenario by In order to meet additional requirement 2, the system simulates a busy situation by launching 6 plane threads to run against 3 gates limit. An emergency landing is revealed by checking for the particular airplane by finding out the hasFuelShortage method. In the main control loop of the ATC, this checks with the landingQueue to determine whether any aircrafts have shortage of fuel.

This approach is justified as one particular aspect of safety of prime importance when dealing with managing priorities. If an emergency is detected, that plane is selected by the controller for immediate landing overriding the other standard practice, to land in first in first out order. This is a demonstration of the balancing of where the normal first come first served thread fairness is broken to safely deal with a critical situation. It is effective in avoiding any thread starvation to emergency aircraft and ensures that the high-risk situation at the time of high congestion in airport terminals is resolved positively.

```java
public class Main {
    /**
     * @param args the command line arguments
     */

    public static Runway runway;
    public static Airport airport;
    public static ATC atc;
    public static RefuelingTruck refuelingTruck;
    public static Gate[] gates;
    public static Statistic statistics;

    // Arrays for the cleaning teams
    public static Cleaning[] cleaners;
    public static Thread[] cleanerThreads;

    private static final int gatesCount = 3;
    private static final int numPlanes = 6;
    private static final int emergencyPlaneId = 5;
```

Figure 24 Congested Scenario Configuration

```java
        // planes 6
        Thread[] planeThreads = new Thread[numPlanes];
        Random rand = new Random();
        for (int i = 1; i <= numPlanes; i++) {
            boolean isEmergency = (i == emergencyPlaneId);
            Plane plane = new Plane(isEmergency);
            Thread planeThread = new Thread(plane, "Plane-" + i);
            planeThreads[i - 1] = planeThread;
            planeThread.start();
            Thread.sleep(rand.nextInt(2001));
        }
```

Figure 25 Plane Thread Initialization with Emergency Flag

```java
while (landingQueue.isEmpty() && takeoffQueue.isEmpty() && running) {
    atcLock.wait();
}
if (!running) break;

if (!landingQueue.isEmpty()) {
    for (Plane p : landingQueue) {
        if (p.hasFuelShortage()) {
            landingPlane = p;
            break;
        }
    }
    if (landingPlane == null) {
        landingPlane = landingQueue.getFirst();
    }
}
```

Figure 26 Emergency Landing Priority Selection Logic

4.0 Conclusion

All the basic and additional requirements have been successfully realised without omission. The system is successfully takes care of the single runway, the 3-plane capacity limitation, and sequential lifecycles. By making use of utilities such as ReentrantLock, Semaphore and synchronized blocks, the architecture is actively avoiding the race condition, starvation and deadlocks resulting in a robust system that efficiently handles the processing of concurrent events within a stringent 60 second window.

5.0 Appendix

```
RefuelingTruck: Online and waiting.
Plane-1: Requesting Landing.
ATC: Landing Permission Granted for Plane-1
Plane-1: Landing.
Plane-2: Requesting Landing.
ATC: Landing Permission Denied for Plane-2. Runway Occupied.
Plane-1: Landed.
Plane-1: Coasting to Gate-1
ATC: Landing Permission Granted for Plane-2
Plane-2: Landing.
Plane-3: Requesting Landing.
ATC: Landing Permission Denied for Plane-3. Runway Occupied.
Plane-1: Docked at Gate-1
Plane-1's Passengers: Disembarking 8 passengers...
Plane-4: Requesting Landing.
Plane-1: Requesting Cleaning at Gate-1
Plane-1: Request refueling.
CleaningTeam-1: Started cleaning Plane-1
RefuelingTruck: Processing Plane-1
Plane-2: Landed.
Plane-2: Coasting to Gate-2
ATC: Landing Permission Granted for Plane-3
Plane-3: Landing.
ATC: Landing Permission Denied for Plane-4. Airport/Gates Full.
Plane-2: Docked at Gate-2
Plane-2's Passengers: Disembarking 41 passengers...
Plane-5: Requesting Landing. [EMERGENCY]
ATC: Landing Permission Denied for Plane-5. Airport/Gates Full.
Plane-3: Landed.
Plane-3: Coasting to Gate-3
Plane-3: Docked at Gate-3
Plane-3's Passengers: Disembarking 33 passengers...
CleaningTeam-1: Finished cleaning Plane-1
RefuelingTruck: Finished Plane-1
Plane-1: Refueling completed.
Plane-1's Passengers: Boarding 36 new passengers...
Plane-6: Requesting Landing.
Plane-2: Requesting Cleaning at Gate-2
Plane-2: Request refueling.
```

Figure 27 Complete Simulation Terminal Output 1

```
CleaningTeam-2: Started cleaning Plane-2
RefuelingTruck: Processing Plane-2
Plane-3: Requesting Cleaning at Gate-3
Plane-3: Request refueling.
CleaningTeam-3: Started cleaning Plane-3
Plane-1: Undock from Gate-1
Plane-1: Coasting to Runway.
Plane-1: Requesting Taking off.
ATC: Takeoff Permission Granted for Plane-1
Plane-1: Taking-off.
CleaningTeam-2: Finished cleaning Plane-2
RefuelingTruck: Finished Plane-2
Plane-2: Refueling completed.
Plane-2's Passengers: Boarding 40 new passengers...
RefuelingTruck: Processing Plane-3
[Main] Plane-1 thread has completed.
ATC: Landing Permission Granted for Plane-5
Plane-5: Landing.
ATC: Landing Permission Denied for Plane-4. Airport/Gates Full.
CleaningTeam-3: Finished cleaning Plane-3
Plane-5: Landed.
Plane-5: Coasting to Gate-1
RefuelingTruck: Finished Plane-3
Plane-3: Refueling completed.
Plane-3's Passengers: Boarding 17 new passengers...
Plane-2: Undock from Gate-2
Plane-2: Coasting to Runway.
Plane-5: Docked at Gate-1
Plane-5's Passengers: Disembarking 20 passengers...
Plane-2: Requesting Taking off.
ATC: Takeoff Permission Granted for Plane-2
Plane-2: Taking-off.
Plane-3: Undock from Gate-3
Plane-3: Coasting to Runway.
Plane-3: Requesting Taking off.
Plane-5: Requesting Cleaning at Gate-1
Plane-5: Request refueling.
CleaningTeam-1: Started cleaning Plane-5
RefuelingTruck: Processing Plane-5
[Main] Plane-2 thread has completed.
```

Figure 28 Complete Simulation Terminal Output 2

```
ATC: Landing Permission Granted for Plane-4
Plane-4: Landing.
ATC: Landing Permission Denied for Plane-6. Airport/Gates Full.
Plane-4: Landed.
Plane-4: Coasting to Gate-2
ATC: Takeoff Permission Granted for Plane-3
Plane-3: Taking-off.
Plane-4: Docked at Gate-2
Plane-4's Passengers: Disembarking 13 passengers...
CleaningTeam-1: Finished cleaning Plane-5
RefuelingTruck: Finished Plane-5
Plane-5: Refueling completed.
Plane-5's Passengers: Boarding 23 new passengers...
[Main] Plane-3 thread has completed.
ATC: Landing Permission Granted for Plane-6
Plane-6: Landing.
Plane-4: Requesting Cleaning at Gate-2
Plane-4: Request refueling.
CleaningTeam-2: Started cleaning Plane-4
RefuelingTruck: Processing Plane-4
Plane-5: Undock from Gate-1
Plane-5: Coasting to Runway.
Plane-6: Landed.
Plane-6: Coasting to Gate-3
Plane-5: Requesting Taking off.
ATC: Takeoff Permission Granted for Plane-5
Plane-5: Taking-off.
Plane-6: Docked at Gate-3
Plane-6's Passengers: Disembarking 37 passengers...
CleaningTeam-2: Finished cleaning Plane-4
RefuelingTruck: Finished Plane-4
Plane-4: Refueling completed.
Plane-4's Passengers: Boarding 24 new passengers...
Plane-4: Undock from Gate-2
Plane-4: Coasting to Runway.
Plane-6: Requesting Cleaning at Gate-3
Plane-6: Request refueling.
CleaningTeam-3: Started cleaning Plane-6
RefuelingTruck: Processing Plane-6
```

Figure 29 Complete Simulation Terminal Output 3

```
Plane-4: Requesting Taking off.
ATC: Takeoff Permission Granted for Plane-4
Plane-4: Taking-off.
[Main] Plane-4 thread has completed.
[Main] Plane-5 thread has completed.
CleaningTeam-3: Finished cleaning Plane-6
RefuelingTruck: Finished Plane-6
Plane-6: Refueling completed.
Plane-6's Passengers: Boarding 26 new passengers...
Plane-6: Undock from Gate-3
Plane-6: Coasting to Runway.
Plane-6: Requesting Taking off.
ATC: Takeoff Permission Granted for Plane-6
Plane-6: Taking-off.
[Main] Plane-6 thread has completed.
Thread-ATC: Shutdown
RefuelingTruck: Shutdown. Total planes: 6
Thread-CleaningTeam-1: Shutdown
Thread-CleaningTeam-2: Shutdown
Thread-CleaningTeam-3: Shutdown
Sanity Check Passed: true
SIMULATION STATISTICS
Waiting Time Statistic:
Max:8784
Min:7
Avg:3804.2
Service Statistics:
Total planes served: 6
Total passengers boarded: 166
Average passengers per plane: 27.7
BUILD SUCCESSFUL (total time: 21 seconds)
```

Figure 30 Complete Simulation Terminal Output 4